

Lecture 27: Analyzing Learning-enabled Cyber-physical systems

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

[ECE484: Principles of Safe Autonomy](#) by Sayan Mitra at UIUC

Other relevant courses on Trustworthy Machine Learning:

[CS 329T](#) at Stanford

[CS 521](#) at UIUC

[ECE1784H/CSC2559H](#) at U Toronto

Recap: What is *trustworthy* machine learning?

- Techniques for evaluating *trustworthiness* of ML models and learning *trustworthy* models
- Metrics of *trustworthiness*:
 - Accuracy
 - Robustness
 - Interpretability
 - Alignment with human values
 - Privacy
 - Being well-calibrated
 - Fairness
 - ...

Two perspectives

Top-down



**System-level
Analysis**

**DNN
Analysis**

Bottom-up



Modern cyber-physical systems

- Reactive systems with deep neural networks (DNNs) for perception (and/or control)
- Vision-based perception with high-dimensional inputs using complex DNNs (millions to billions of parameters)
- Safety-critical settings



Autonomous cyber-physical systems can fail!

Tesla behind eight-vehicle crash was in 'full self-driving' mode, says driver

San Francisco crash is the latest in a series of accidents blamed on Tesla technology, which is facing regulatory scrutiny

BAY AREA // SAN FRANCISCO

Driver hits woman in S.F., then Cruise driverless car runs her over; photo shows victim trapped

By Jordan Parker, Nora Mishanec

Updated Oct 3, 2023 3:52 p.m.



BAY AREA

Waymo driverless taxi hits bicyclist in San Francisco

By Jordan Parker

Updated Feb 6, 2024 7:27 p.m.



US agency probes Amazon-owned Zoox self-driving vehicles after two crashes

By David Shepardson

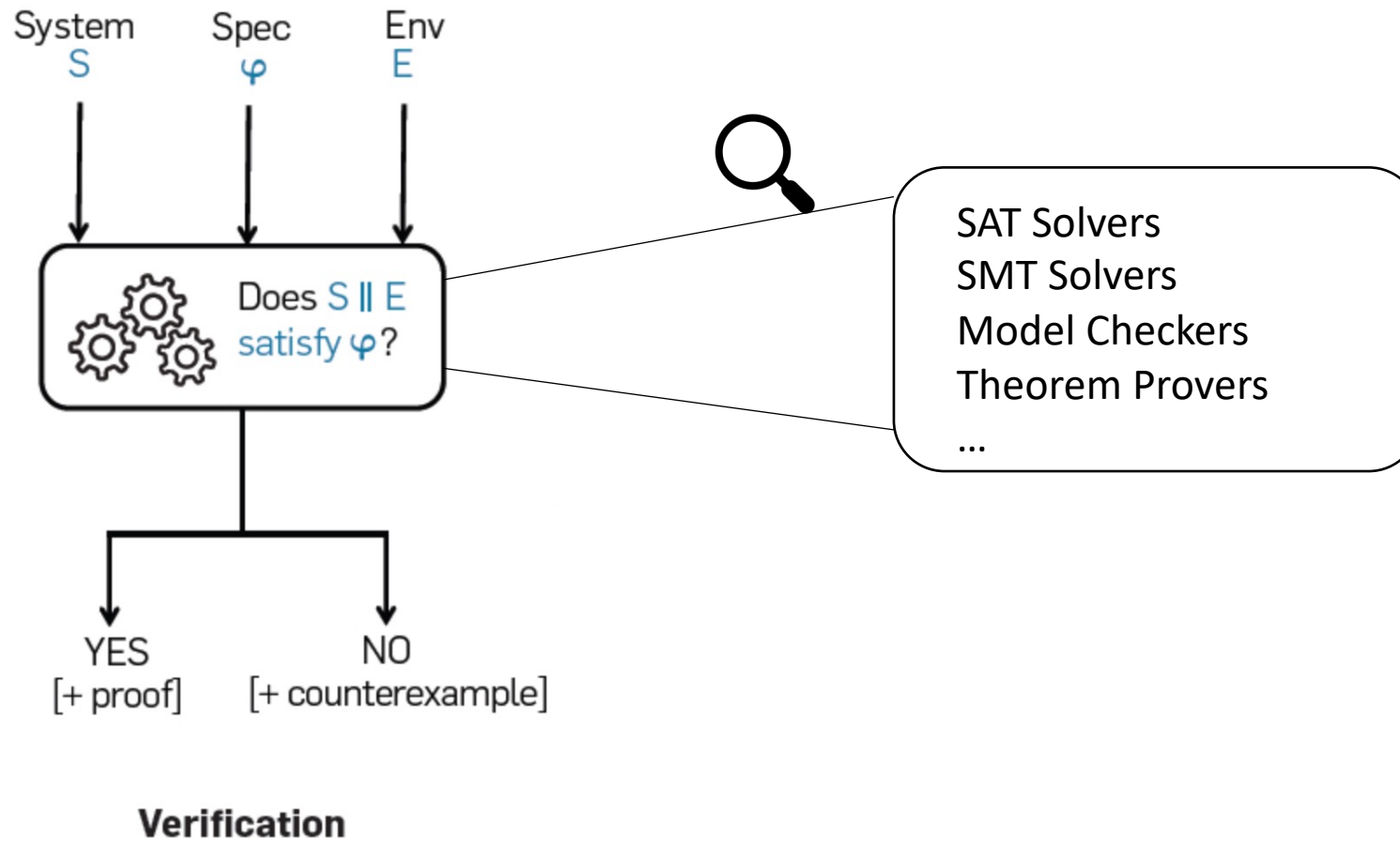
May 13, 2024 8:44 AM PDT · Updated 7 hours ago



Naively collecting test driving miles is also not going to work!

- Probability of a fatal accident per one hour of human driving: 10^{-6}
- Assume that for AV this has to be 10^{-9}
- Data required to guarantee a probability of 10^{-9} fatality per hour of driving is inversely proportional: 10^9 hours, 30 billion miles
- Multi-agent, open system, with human interactions => cannot be simulated offline to generate data
- Any change in software means tests have to be rerun

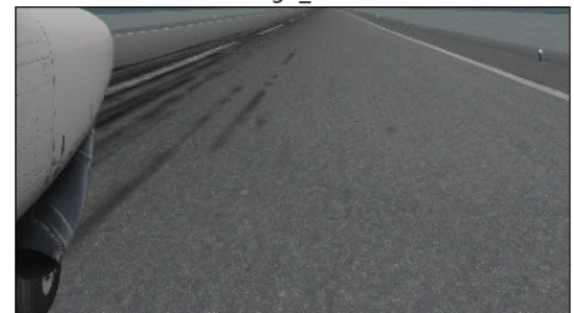
Safety analysis via formal methods



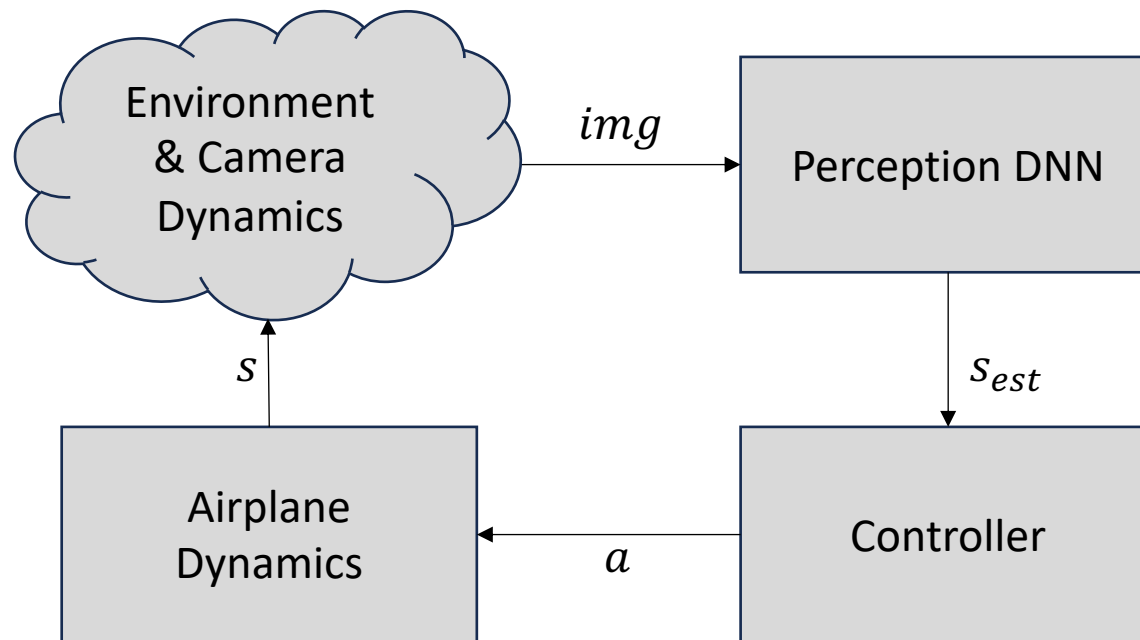
Closed-loop Analysis of Vision-based Autonomous Systems: A Case Study by Pasareanu et al.

Case study: TaxiNet

- Autonomous aircraft taxiing system
- Tracks center-line of the taxiway
- DNN gets picture of the taxiway as input and estimates the plane's state w.r.t. the center-line
- Returns two numerical outputs
 - Cross-track error (**cte**): The distance of the plane from the center-line
 - Heading error (**he**): The angle of the plane w.r.t. the center-line
- Hand-written controller from estimated system state (**cte**, **he**) to steering angle



Case study: TaxiNet



Discretized view of TaxiNet

- Discrete-state model of the system for analysis
 - Discrete controller
 - Discrete model of airplane dynamics
- Regression outputs of DNN are discretized to view it as a *classifier*

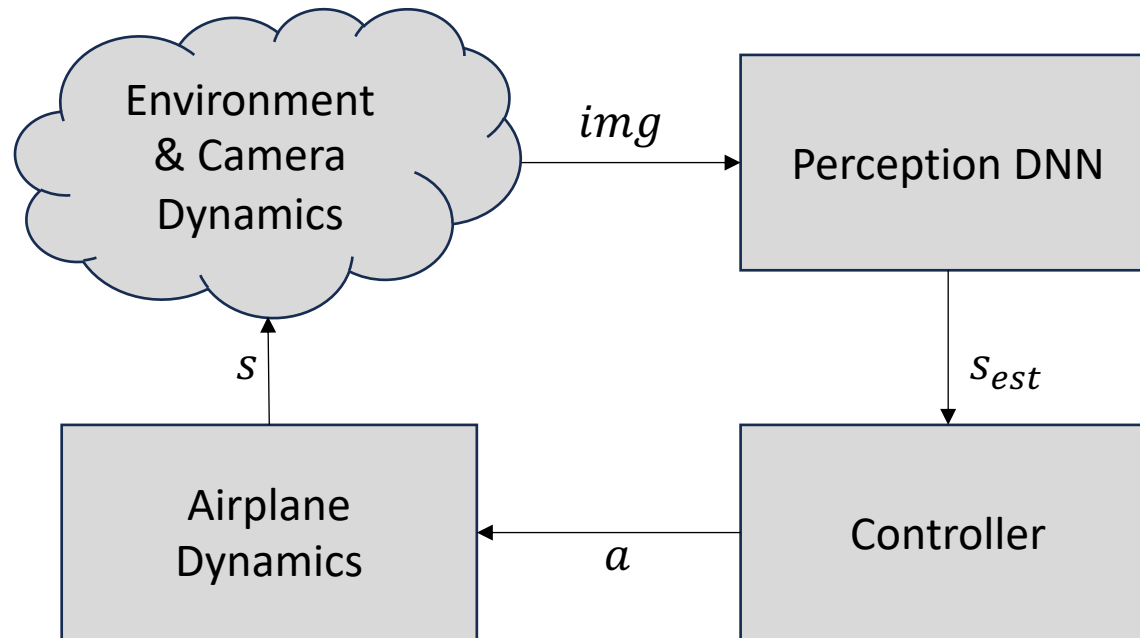
$$\underline{\text{cte}} = \begin{cases} 3 & \text{if } -8.0 \text{ m} \leq \text{cte} \leq -4.8 \text{ m} \\ 1 & \text{if } -4.8 \text{ m} \leq \text{cte} \leq -1.6 \text{ m} \\ 0 & \text{if } -1.6 \text{ m} \leq \text{cte} \leq 1.6 \text{ m} \\ 2 & \text{if } 1.6 \text{ m} \leq \text{cte} \leq 4.8 \text{ m} \\ 4 & \text{if } 4.8 \text{ m} \leq \text{cte} \leq 8.0 \text{ m} \end{cases}$$

$$\underline{\text{he}} = \begin{cases} 1 & \text{if } -35.0^\circ \leq \text{he} \leq -11.67^\circ \\ 0 & \text{if } -11.67^\circ \leq \text{he} \leq 11.66^\circ \\ 2 & \text{if } 11.66^\circ \leq \text{he} \leq 35.0^\circ \end{cases}$$

TaxiNet: Safety specifications

- Simple scenario:
 - From a fixed initial state, ensure safe operation on a straight segment of the taxiway for a finite number of steps
 - Safe operation: $|\mathbf{cte}| \leq 8$ meters and $|\mathbf{he}| \leq 35$ degrees
- Formalized as probabilistic temporal logic (PCTL) queries:
 - (*Property 1*) Airplane goes off runway: $\mathbf{P = ?[F (|cte| > 8)]}$
 - (*Property 2*) Airplane turns more than certain degree: $\mathbf{P = ?[F (|he| > 35)]}$

How to analyze TaxiNet?



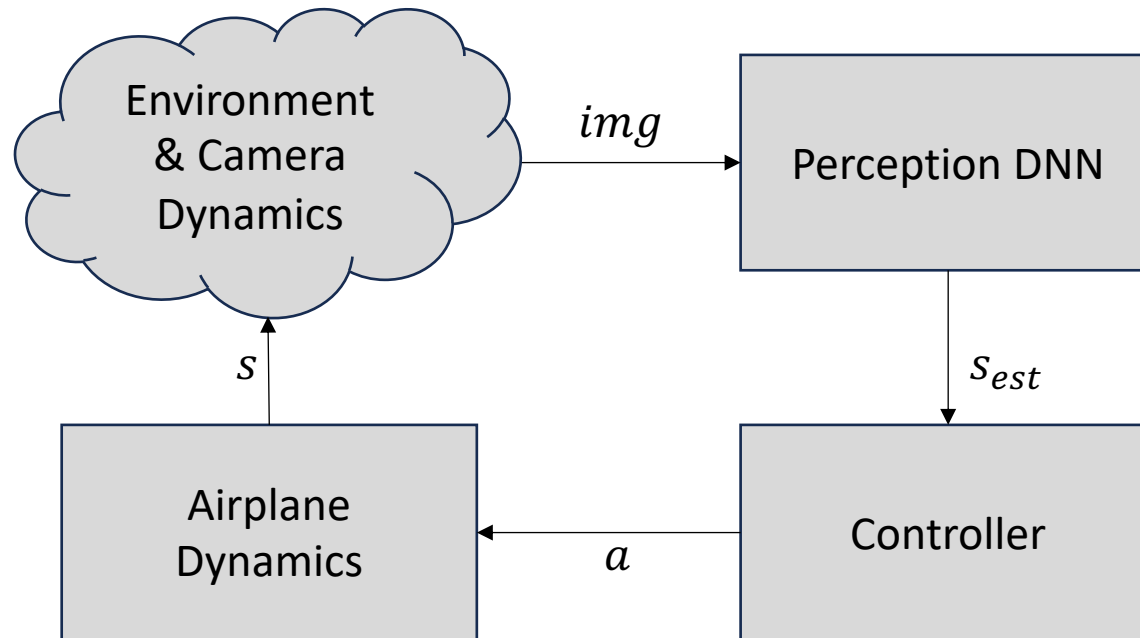
How to analyze discrete cyber-physical systems?

- Mathematically model system components as *transition systems*
- Transition system: Theoretical device to study the evolution of system and prove things about its behavior
 - Consists of a set of states S , set of initial states $S_0 \subseteq S$, and a transition relation $\rightarrow \subseteq S \times S$
 - Reachable states: All states “reachable” (via transition relation) from the set of initial states
- Model safety property as the set of “unsafe” states
- Prove that unsafe states are not reachable from the initial states

Analyzing safety of autonomous systems is hard 😞

- DNNs are complex and opaque
- DNNs lack logical specifications
- Operating environments are too complex to be mathematically modeled
- Formal methods do not scale

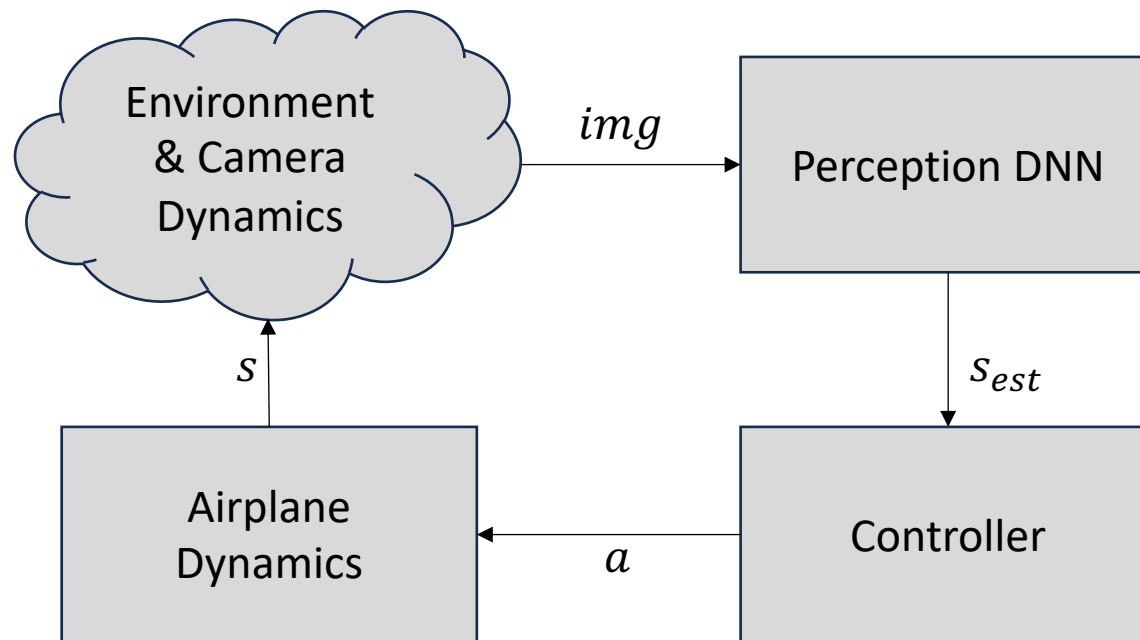
How to analyze TaxiNet?



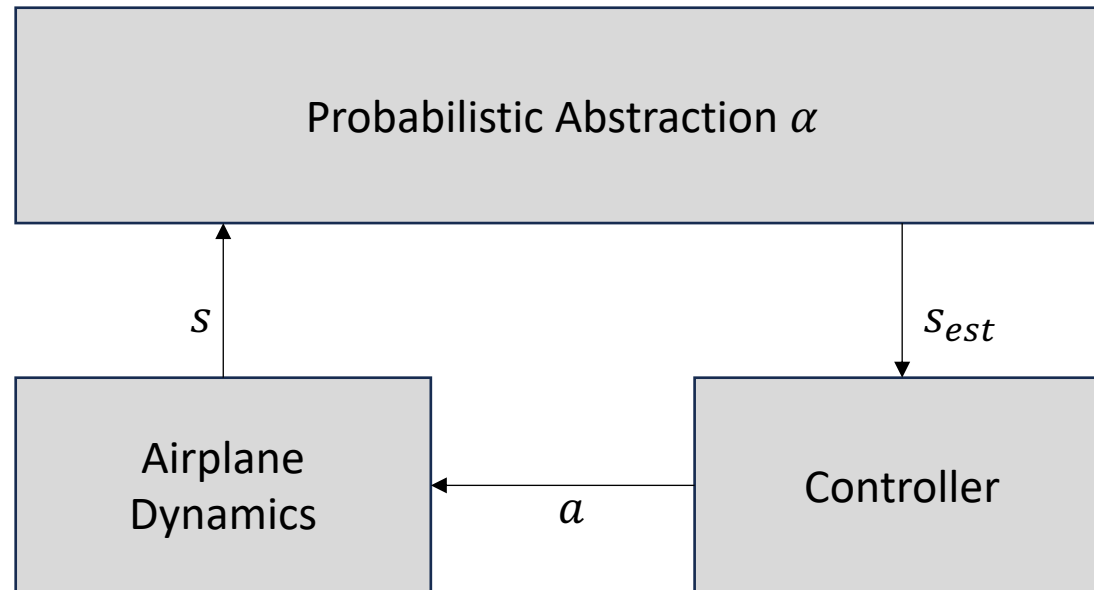
DNN too large

Mathematical models of
environment & camera too
complex

Idea



Idea



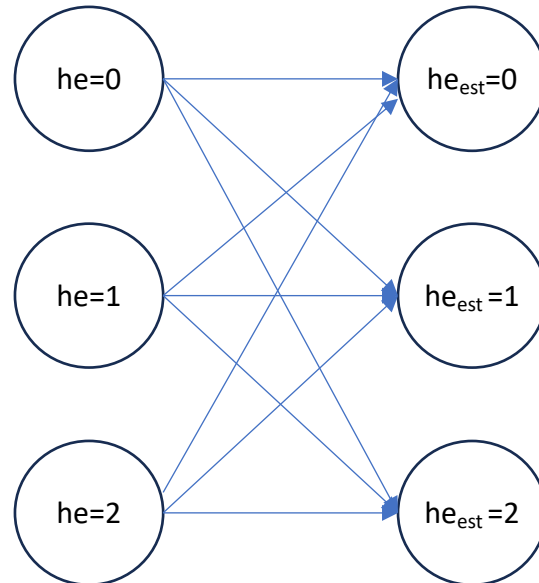
Probabilistic map from **actual** system states s to (distribution over) **estimated** states s_{est}

Why probabilistic?

- Environment is stochastic, i.e., each airplane state maps to a distribution over images
- DNN is imperfect

Probabilistic abstraction α

- Probabilistic transition from every (discrete) actual system state to every (discrete) estimated state
- Transition probabilities estimated based on confusion matrices for perception DNN, measured on “**representative**” data set



		Predicted			
		Total = 11108	0	1	2
Actual	0	4748	2139	148	
	1	91	2010	0	
	2	744	211	1017	

Table 1: Confusion Matrix for he

$$\Pr(he_{est} = 1 \mid he = 0) = \frac{2139}{4748 + 2139 + 148}$$

Probabilistic abstraction α

- α **quadratic** in the size of the DNN output and independent of the number of DNN parameters, the camera or the environment
- Closed-loop system modeled as a Discrete-time Markov Chain (DTMC)
- Amenable for verification with probabilistic model checking tools

Improving safety via run-time guards

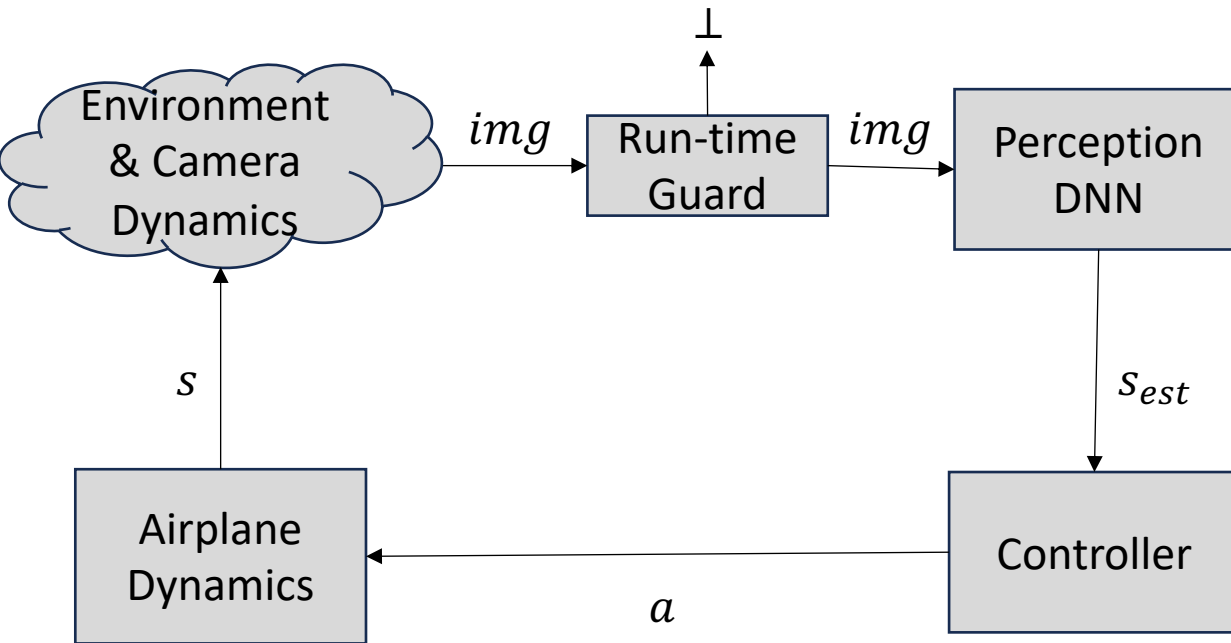
Several techniques for analysis of DNNs in isolation:

- Local robustness certifiers
- Out-of-distribution detectors
- ...

How can we leverage these DNN-specific analyses to improve the safety of the overall system?

As run-time guards!

DNN checks as run-time guards



- Use existing DNN checks as run-time guards
- On inputs where check passes, DNN is more trustworthy and accurate
- If the check is violated M times, go to “abort” state (e.g., hand over control to the pilot)
- Compute confusion matrix only for inputs that pass the check

Experiments

Analyzed two system models:

- m1 (no run-time guard)
- m2 (with run-time guard)
 - Rules of the form **Pre** => **Post** characterizing inputs where the DNN mis-predicts
 - Pre is a condition over intermediate neurons; Post is a condition on the output
 - An input passes the guard if it does not satisfy Pre

Controller and dynamics are the same for both models

Logical rules as run-time guards

- For case study, we use logical rules of form $\text{Pre} \Rightarrow \text{Post}$ extracted automatically from DNN
- Pre is a condition on neuron values at some layer; Post = “mis-prediction”

$$\begin{aligned} &N_{1,85} \leq -0.998 \wedge N_{2,50} \leq 3.31 \wedge N_{1,84} \leq -0.994 \wedge N_{1,15} > -0.999 \\ &\wedge N_{1,21} \leq 1.711 \wedge N_{1,70} \leq 11.088 \wedge N_{1,51} > -0.999 \wedge N_{1,21} > -0.637 \implies \\ &|\text{cte}^* - \text{cte}| > 1.0 \text{ meters} \vee |\text{he}^* - \text{he}| > 5 \text{ degrees} \end{aligned}$$

- If an input satisfies Pre it is considered to violate the runtime check
- Can be evaluated efficiently during forward pass of DNN

TaxiNet: Safety specifications

(Property 1) Airplane does not go off runway:

$$\mathbf{P = ?[F (|cte| > 8)]}$$

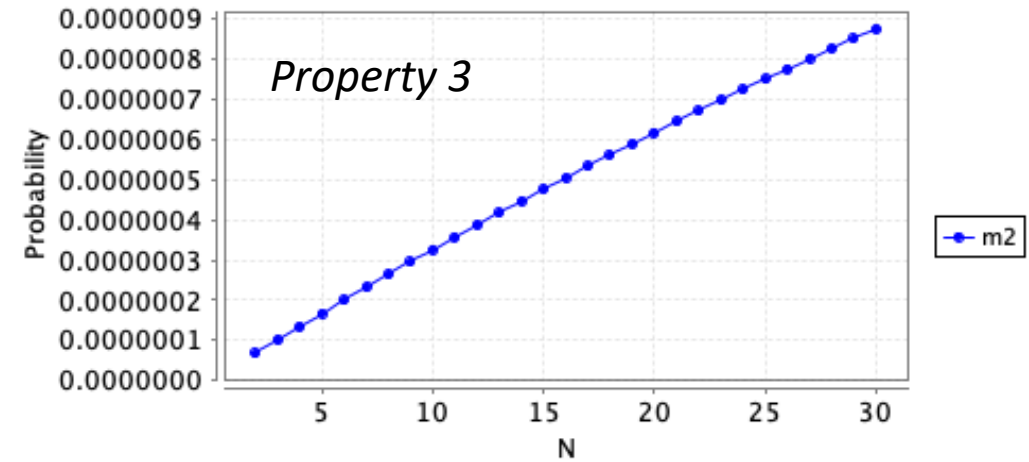
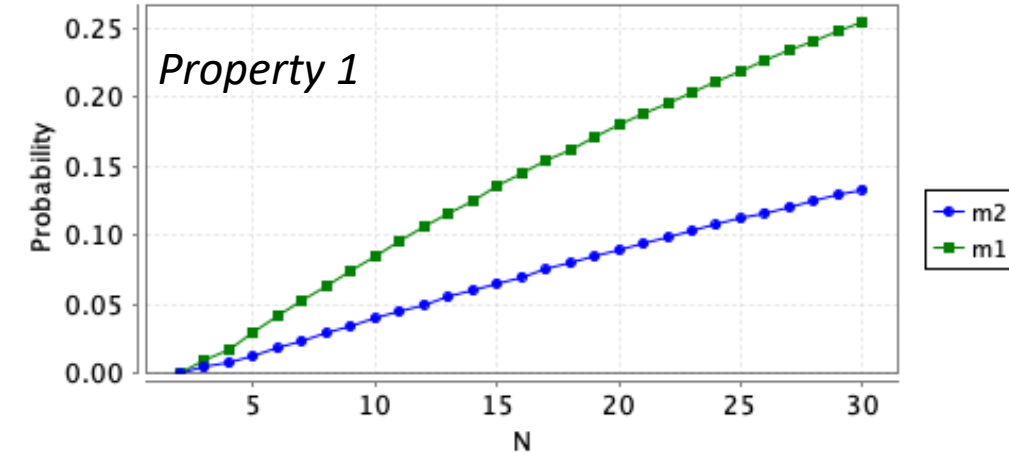
(Property 2) Airplane does not turn more than certain degree:

$$\mathbf{P = ?[F (|he| > 35)]}$$

(Property 3) TaxiNet system does not reach abort state:

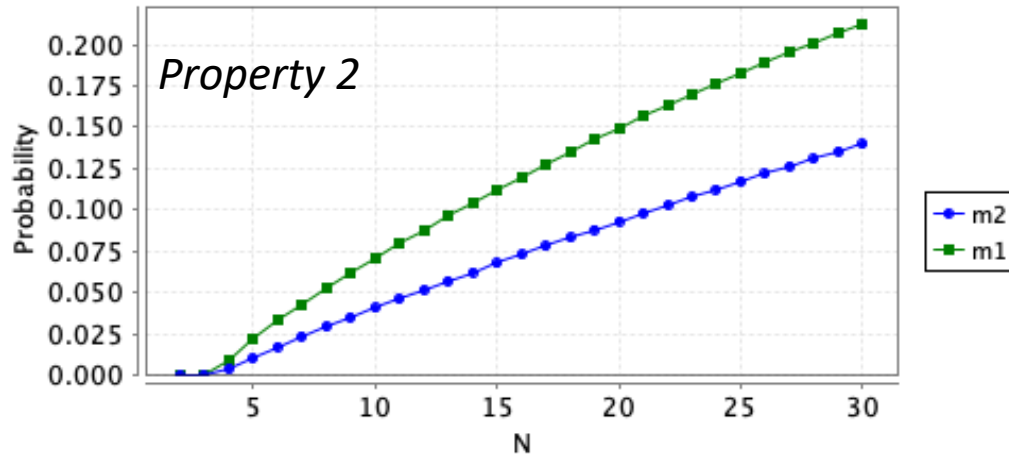
$$\mathbf{P = ?[F (\mathbf{v} = 0 \ \& \ i = M)]}$$

Results: Safety analysis



Results calculated using PRISM

Average running time < 0.1 seconds
on an M1 MacBook Pro, 16GB RAM

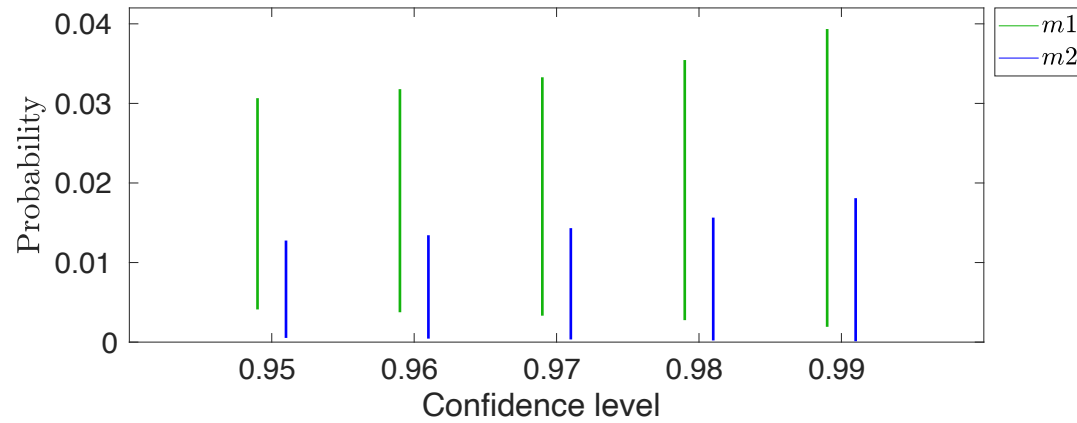


Confidence interval analysis

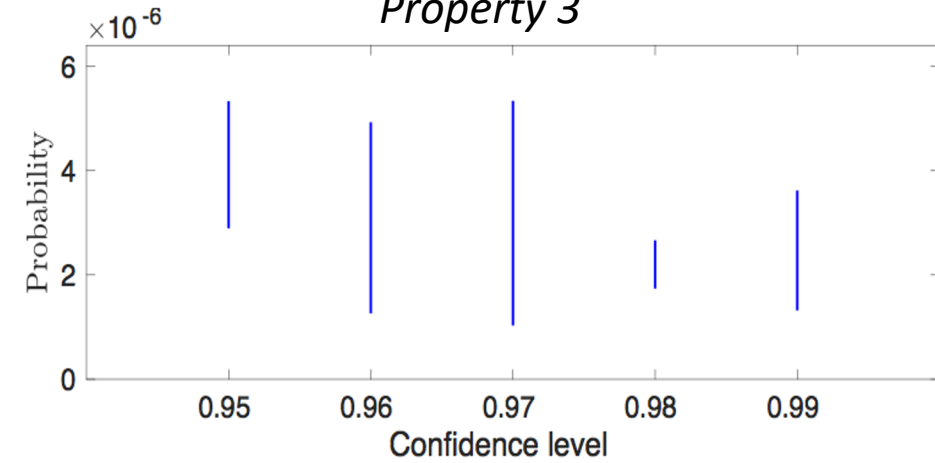
- Probabilistic abstractions relies on calculating empirical point estimates of the required probabilities.
- These empirical estimates lack **statistical guarantees**; can be off by an arbitrary amount from the true probabilities.
- We compute **confidence intervals** based on confusion matrices
 - For the transition probabilities
 - For the probability that the safety properties are satisfied.
- FACT tool [Calinescu et al., TACAS'16]:
 - Given parametric DTMC m , PCTL property ϕ , error level $\delta \in (0, 1)$, *number of observations* for each parametric transition
 - FACT synthesizes a $(1 - \delta)$ -**confidence interval** $[a, b] \subseteq [0, 1]$ for the probability that ϕ is satisfied, given the observations

Results: Confidence interval analysis

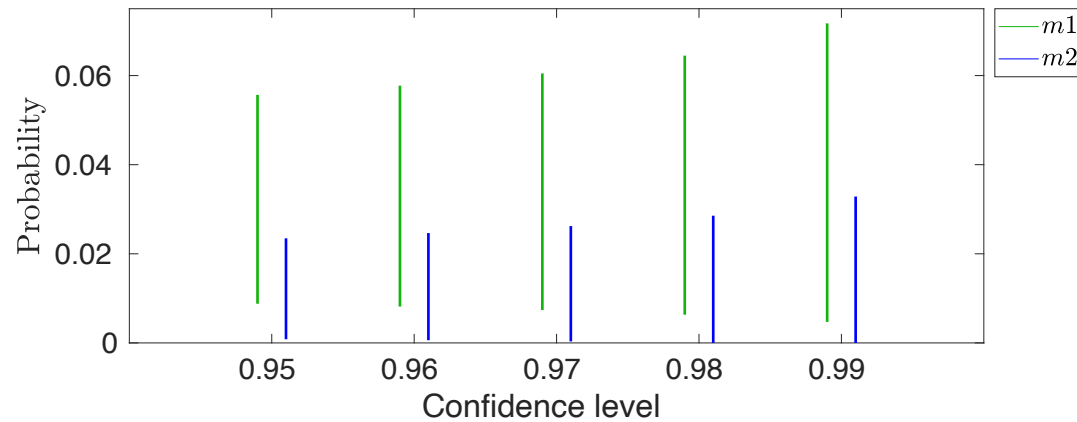
Property 1



Property 3



Property 2



Takeaways

- Approach makes analysis feasible
- Testing artifacts like confusion matrices can be fruitfully employed for system-level analysis
- Abstraction enables compositional system development and reasoning
 - Best-case analysis
 - Average-case analysis
 - Worst-case analysis
- DNNs are more accurate on inputs that pass run-time checks
- Run-time guards improve system safety

